2026-5-31 C++AVL树详解

- 1. AVL树的概念
- 2. AVL树的实现
  - 2.1 AVL树的结构
  - 2.2 AVL树的插入
    - 2.2.1 平衡因子的更新原则
    - 2.2.2 平衡因子更新的步骤
    - 2.2.3 插入结点以及更新平衡因子的代码实现
- 3. 旋转
  - 3.1 旋转原则
  - 3.2 单旋
    - 3.2.1 右单旋（顺着的右边失衡）
    - 3.2.2 左单旋
  - 3.2 双旋
    - 3.2.1 左右双旋
    - 3.2.2 右左双旋：
- 4. 检查是否平衡的函数
- 5.查找函数
- 6. 最后测试一下

1. AVL树的概念

AVL树是一棵具有如下性质的二叉搜索树：

1. 它可以是一棵空树也可以是一棵左右子树都是AVL树的平衡二叉树，且左右子树的高度不超过1（也就是任意一棵子树的左右高度差为-1，0，1的平衡二叉树）
2. AVL树是一棵高度平衡的二叉搜索树，这就意味着它就没有二叉搜索树查找效率最坏的（单支树）的情况，它可以稳定的维持$\log_2(N)$的查找效率。
3. 因为我们需要控制AVL树任意一棵子树的左右子树高度差不超过1（也就是只有-1，0，1这三种情况了，因为树的结点个数可能是奇数也可能是偶数，是没有办法保持高度差一直为0的，维持-1，0，1这三种已经是最好的情况了），那么我们就引入了平衡因子这个概念，在每个节点旁写平衡因子并不是必须的，但是写上可以帮助我们对该AVL树进行判断。
4. AVL树的高度可以控制在log_N,增删改查的效率控制在0(log_N)。

2. AVL树的实现

2.1 AVL树的结构

1. 首先我们需要建立四个变量，pair<K,V>的存储键值对，除此之外还要分别定义指向`_left`,`_right`，和`_parent`的指针变量,注意这里还要定义一个平衡因子
2. 构造函数构造初始化该结点结构
3. 定义类模板,私有成员变量`_root`

```
template<class K,class V>
struct AVLTreeNode
{
    pair<K,V> _kv;
    AVLTreeNode<K,V>* _right;
    AVLTreeNode<K,V>* _left;
    AVLTreeNode<K,V>* _parent;
    //balence factor
    int _bf;
    AVLTreeNode(const pair<K,V>& kv)
        :_kv(kv)
        ,_right(nullptr)
        ,_left(nullptr)
        ,_parent(nullptr)
        ,_bf(0)
    { }
};

template<class K, class V>
class AVLTree {
    typedef AVLTreeNode<K,V> Node;
public:
private:
    Node* _root = nullptr;
};
```

2.2 AVL树的插入

AVL树的插入比二叉平衡树插入会多在平衡因子的更新和旋转处理。大致过程如下：
第一步：将一个值按照二叉搜索树的规则进行插入
第二步：新增结点后会影响部分祖先的平衡因子，所以这个时候我们需要更新从新增结点->根节点路径上的平衡因子。如果平衡因子不符合规则，就还需要对该树进行旋转，具体流程如下：

2.2.1 平衡因子的更新原则

1. 首先平衡因子=左子树的高度-右子树的高度
2. 只有子树的高度发生变化才会影响其祖先的平衡因子
3. 如果你的平衡因子是按照左子树-右子树的方式计算的，那么可以很轻松的推理得出如果在左子树的高度+1，则平衡因子++；如果在右子树的高度+1，则平衡因子--
4. 更新平衡因子结束后，所有结点的平衡因子维持在小于一的情况，则插入结束。
5. 如果出现平衡因子为2或者-2的情况，则该AVL树失衡，需要进行旋转。

> 在这里可以来思考一个问题：出现失衡情况是为什么更新的平衡因子不会出现3，-3，4，-4.....的情况呢?
> 因为如果出现这样的情况，在你插入数据之前你的AVL树就是失衡的，如果你之前AVL树是平衡的是不会出现这样的情况的。

2.2.2 平衡因子更新的步骤

1. 插入结点元素后，如果插入parent的平衡因子由-1->0或者由1->0，那么说明你更新前parent子树一边高一边低，新增的结点插入到了低的那边。这个时候说明所在子树的高度没有发生变化，平衡因子的变化不会向上蔓延，更新结束。
2. 插入结点元素后，平衡因子由0->-1或者0->1，那么说明你更新前parent的子树两边一样高，新增的结点插入后parent所在的子树一边高一边低，这个时候所在子树的高度+1发生了变化，平衡因子的变化就会向上蔓延变化，平衡因子向上更新。
3. 插入结点元素后平衡因子由-1->-2或者由1->2，那么说明你在更新前parent的子树一边高一边低，但是你插入高的那一头这个时候所在子树的高度+1发生了变化，同时该子树的平衡被破化了，我们需要对该子树进行旋转。旋转的目的有两个：

- 把parent子树旋转平衡
- 降低parent子树的高度，恢复到插入结点以前的高度。所以**旋转后平衡因子不需要向上更新，插入结束**。

2.2.3 插入结点以及更新平衡因子的代码实现

插入结点过程：

1. 传入需要插入的数据，首先判断根结点是否为空，为空则将该结点置为根结点。
2. 根结点不为空，遍历搜寻插入位置，使用cur判断以及标记插入的位置，同时记录cur的parent便于插入数据。
3. 以cur不为空为循环条件，判断cur结点的first与插入值。如果大于向右移动，如果小于向左移动，否则返回false。

4. 查找到插入位置，将数据进行插入，new结点并将结点连接到parent的合适的位置。将cur的parent赋值为parent（这个别忘了，你比二叉平衡树多了一个parent结点）
   更新平衡因子过程：
5. 判断条件为parent不为空，如果cur是parent的左孩子，平衡因子++，为右孩子，平衡因子--（平衡因子=左子树的高度-右子树的高度）
6. 判断parent平衡因子的值决定平衡因子是向上蔓延，还是不动，还是将树进行旋转。
7. 如果平衡因子为0更新结束跳出循环
8. 平衡因子为1或者-1平衡因子向上更新
9. 平衡因子为2或者-2失衡旋转处理
10. 都不是断言报错，说明之前给的二叉平衡树是错误的。
    具体代码实现：

```
// 3. 更新平衡因子并旋转
while (parent) {
    if (cur == parent->_left) {
        parent->_bf--;
    }
    else {
        parent->_bf++;
    }

    if (parent->_bf == 0) {
        break;
    }
    else if (parent->_bf == 1 || parent->_bf == -1) {
        cur = parent;
        parent = parent->_parent;
    }
    else if (parent->_bf == 2 || parent->_bf == -2) {
        // 旋转处理
        if (parent->_bf == -2 && cur->_bf == -1) {
            RotateR(parent);
        }
        else if (parent->_bf == 2 && cur->_bf == 1) {
            RotateL(parent);
        }
        else if (parent->_bf == -2 && cur->_bf == 1) {
            RotateLR(parent);
        }
        else if (parent->_bf == 2 && cur->_bf == -1) {
            RotateRL(parent);
        }
        else {
            assert(false);
```

```
            }
            break;
        }
```

3. 旋转

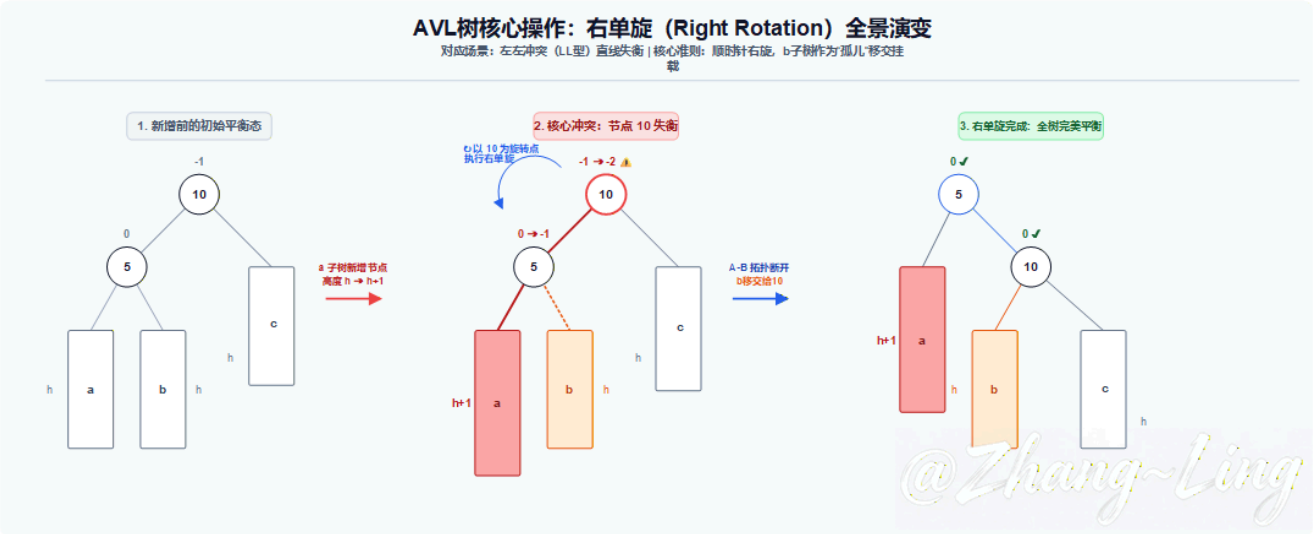
3.1 旋转原则

1. 保持搜索树的规则
2. 使得二叉平衡树由不平衡变得平衡，其次降低旋转树的高度。

3.2 单旋

3.2.1 右单旋（顺着的右边失衡）

图中展示的10为根的树，a/b/c抽象为三棵高度为h的子树（h>=0),a/b/c均符合AVL树的要求。



右单旋的代码思路：

1. 定义两个变量subL为parent的左孩子，subLR为subL的左孩子的右孩子
2. 让subLR成为parent的右孩子，如果subLR不为空，则让subLR的parent指向parent
3. 让parent成为subL的右孩子，改变parent的parent的指向为subL这个结点。
4. 即让subL成为parent，还要让原本parent的parent指向subL,所以在修改parent的parent之前要将该结点信息保留下来。
5. 如果说parent是根节点，那么就将根节点替换成subL,并将其parent置为空。
6. 如果不是则判断其是原本parent的parent的左孩子还是右孩子，然后将其parent置为原本parent的parent。

```
// 右单旋
void RotateR(Node* parent) {
    Node* subL = parent->_left;
    Node* subLR = subL->_right;

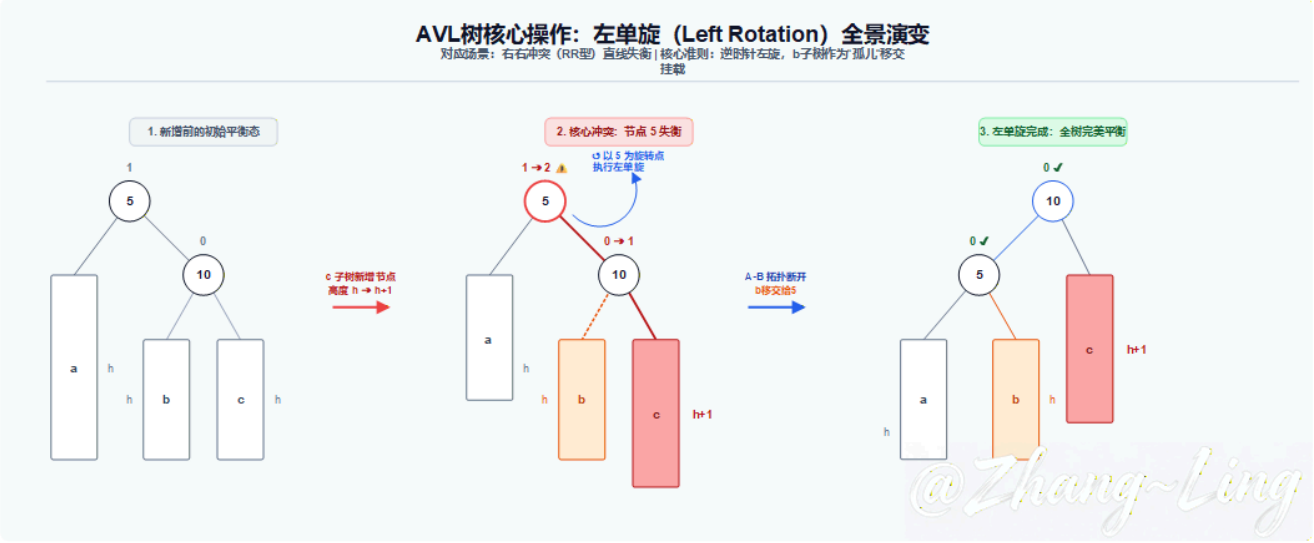
    parent->_left = subLR;
    if (subLR) subLR->_parent = parent;

    subL->_right = parent;
    Node* parentparent = parent->_parent;
    parent->_parent = subL;

    if (parentparent == nullptr) {
        _root = subL;
        _root->_parent = nullptr;
    }
    else {
        if (parentparent->_left == parent) {
            parentparent->_left = subL;
        }
        else {
            parentparent->_right = subL;
        }
        subL->_parent = parentparent;
    }

    parent->_bf = subL->_bf = 0;
}
```

3.2.2 左单旋



代码实现：

1. 首先定义两个两个变量，不平衡的结点为parent，定义subR为parent的右子树；subRL为parent的右子树的左子树
2. 让subRL变成parente的左子树，你定了subRL就连带着要动它的父亲，也就是不止要改变它的指向还要改变其父亲的指向防止subRL为空要做一个判断
3. 左旋让parense指向subR的左边，让其_parent为subR如果之前的parent根，那么subR就要变成新根。他的父亲就要指向空。
4. parent如果不是根就要与上一层连接，记录parentparent连接
5. 最后sbuR和parents的平衡因子都置为0

```
// 左单旋
void RotateL(Node* parent) {
    Node* subR = parent->_right;
    Node* subRL = subR->_left;

    parent->_right = subRL;
    if (subRL) subRL->_parent = parent;

    subR->_left = parent;
    Node* parentparent = parent->_parent;
    parent->_parent = subR;

    if (parentparent == nullptr) {
        _root = subR;
        _root->_parent = nullptr;
    }
    else {
        if (parentparent->_left == parent) {
            parentparent->_left = subR;
        }
        else {
            parentparent->_right = subR;
        }
        subR->_parent = parentparent;
    }

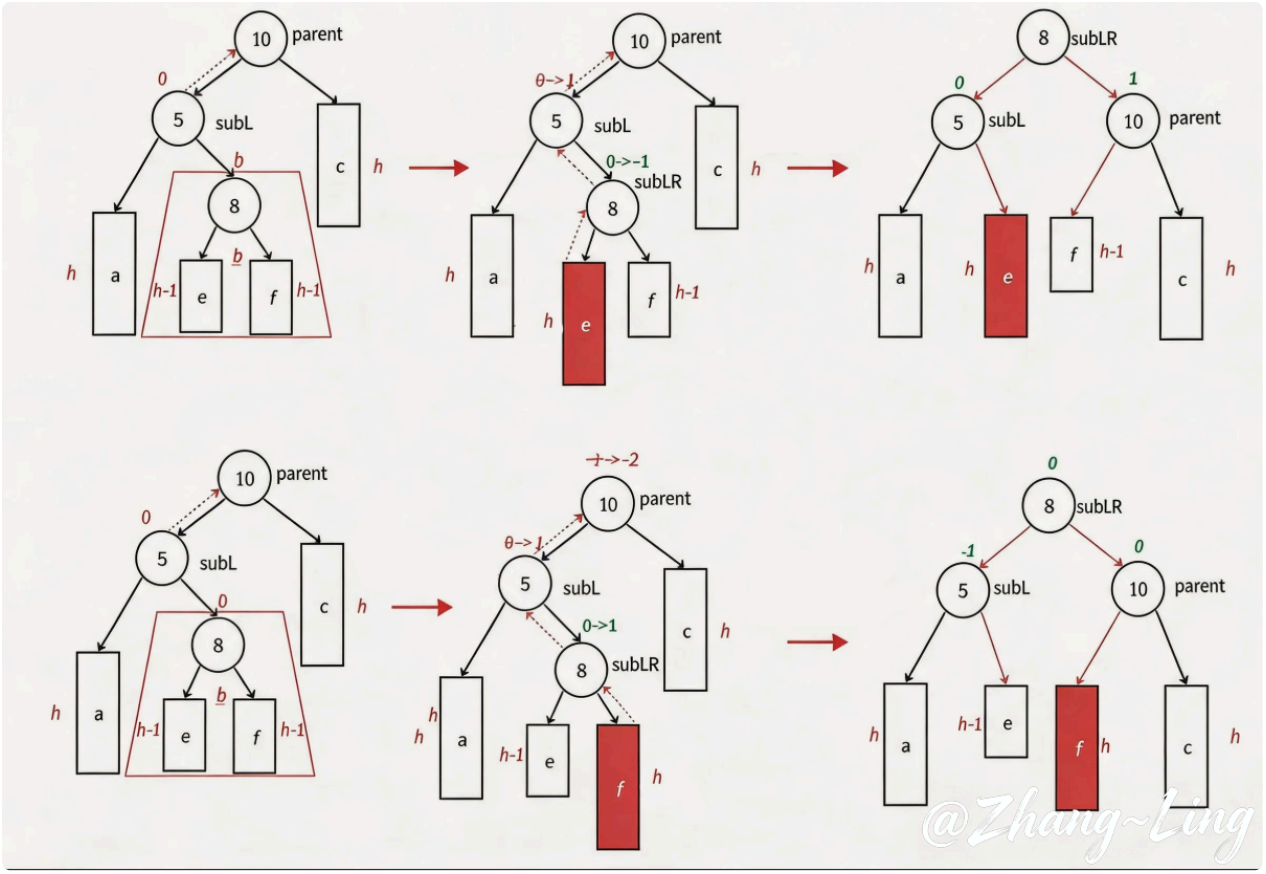
    parent->_bf = subR->_bf = 0;
}
```

3.2 双旋

3.2.1 左右双旋

单旋的本质是纯粹的一边高，如果对于10而言是右边高对于15而言是左边高，高的不纯粹，基于该原因，要让其旋转两次。现在按照以前的方法就会将其从右边高左边高变为左边高右边高，这样显然就不对了，我们就需要旋转两次：

1. 先要将局部进行调整，先对子树的子树进行左单旋转，调整为一边高；再对其进行进一步的拆解，在对另一边进行右边单旋转这样就一边高了。
2. 为什么还要在划分两种情况？因为后面会影响到最后的平衡因子，但是你的旋转逻辑都是一样的，关键区别在于平衡因子不同
   涉及到三种平衡因子的调节，要区分，我们应该如何区分？直接去观察subL的平衡因。



左右双旋的代码实现：

3. 定义变量subL指向parent的左孩子，subRL指向parentsubL的右孩子
4. 记录subLR的平衡因子数目
5. 因为左右双旋发生在左子树失衡但是高度的改变在左子树的右边高度发生了改变，此时就要对该部分先左旋再右旋
6. subL作为parent进行左旋，再对parent进行右边旋
7. 更新平衡因子如果原本的subLR为0，则subL，parent，subLR的平衡因子都为0；如果原本的平衡因子为1，则subL的平衡因子更新为-1，parent和subLR的平衡因子为0；如果原本的平衡因子为-1，则subL的平衡因子为0，parent的平衡因子为1，subLR的平衡因子为0。
8. 如果都不是则断言报错

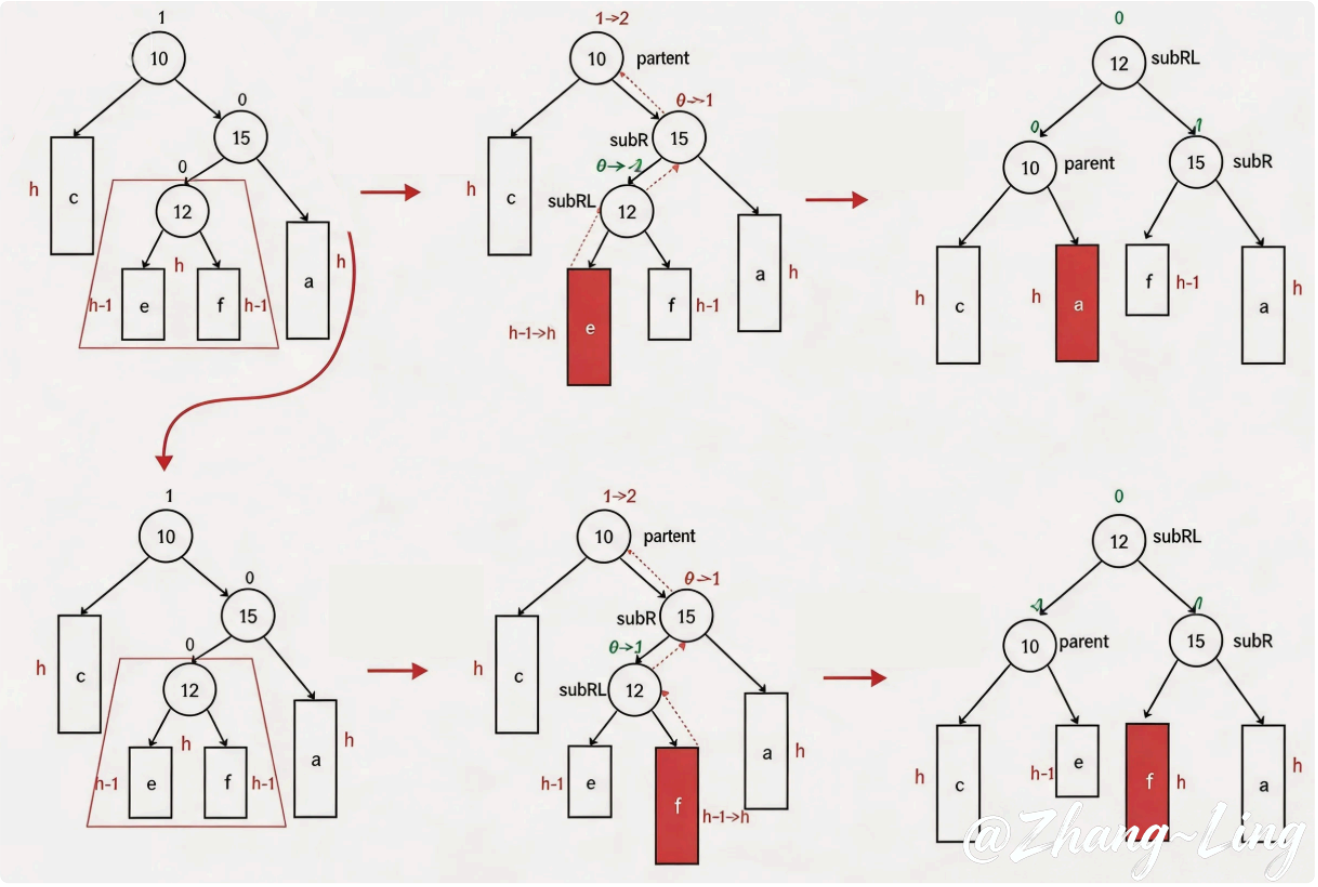
```
// 左右双旋
void RotateLR(Node* parent) {
    Node* subL = parent->_left;
    Node* subLR = subL->_right;
    int bf = subLR->_bf;

    RotateL(subL);
    RotateR(parent);

    if (bf == 0) {
        subL->_bf = 0;
        parent->_bf = 0;
        subLR->_bf = 0;
    }
    else if (bf == 1) {
        subL->_bf = -1;
        parent->_bf = 0;
        subLR->_bf = 0;
    }
    else if (bf == -1) {
        subL->_bf = 0;
        parent->_bf = 1;
        subLR->_bf = 0;
    }
    else {
        assert(false);
    }
}
```

3.2.2 右左双旋：

这个和上面的类似：



右左双悬的代码实现：

1. 定义变量subR指向parent的右孩子，subRL指向subR的左孩子
2. 记录subRL的平衡因子数目
3. 因为左右双旋发生在右子树失衡但是高度的改变在右子树的左边高度发生了改变，此时就要对该部分先右旋再左旋
4. subR作为parent进行左旋，再对parent进行右边旋
5. 更新平衡因子如果原本的subRL为0，则subR，parent，subRL的平衡因子都为0；如果原本的平衡因子为1，则parent的平衡因子更新为-1，sunR和subLR的平衡因子更新为0；如果原本的平衡因子为-1，则subR的平衡因子更新为1，parent和subLR的平衡因子更新为0。

```
// 右左双旋
void RotateRL(Node* parent) {
    Node* subR = parent->_right;
    Node* subRL = subR->_left;
    int bf = subRL->_bf;t

    RotateR(subR);
    RotateL(parent);

    if (bf == 0) {
        subR->_bf = 0;
```

```
        parent->_bf = 0;
        subRL->_bf = 0;
    }
    else if (bf == 1) {
        subR->_bf = 0;
        parent->_bf = -1;
        subRL->_bf = 0;
    }
    else if (bf == -1) {
        subR->_bf = 1;
        parent->_bf = 0;
        subRL->_bf = 0;
    }
    else {
        assert(false);
    }
}
```

4. 检查是否平衡的函数

要检查AVL树是否平衡就是检查左右子树的高度差（写递归后序遍历的话效率会更高一些）：

1. 首先判断根节点是否为空，为空则返回真
2. 整体思路就是分别求出左右子树的高度做差检查，递归分别求出左子树和右子树的高度
3. 左子树-右子树的得出平衡因子是多少并判断，如果平衡因子的绝对值大于等于2，返回该结点的值并输出“平衡因子异常”
4. 如果不符合上述情况则继续递归知道递归到根结点返回true

```
int _Height(Node* root) {
    if (root == nullptr) return 0;
    int leftHeight = _Height(root->_left);
    int rightHeight = _Height(root->_right);
    return max(leftHeight, rightHeight) + 1;
}
 bool _IsBalanceTree(Node* root) {
     if (root == nullptr) return true;

     int leftHeight = _Height(root->_left);
     int rightHeight = _Height(root->_right);
     int bf = rightHeight - leftHeight;

     if (abs(bf) >= 2) {
         cout << root->_kv.first << " 平衡因子绝对值 >= 2" << endl;
```

```
            return false;
        }
        if (bf != root->_bf) {
            cout << root->_kv.first << " 平衡因子异常：实际=" << bf << ", 存储="
<< root->_bf << endl;
            return false;
        }

        return _IsBalanceTree(root->_left) && _IsBalanceTree(root->_right);
    }
```

5.查找函数

```
Node* Find(const K& key) {
    Node* cur = _root;
    while (cur) {
        if (key > cur->_kv.first) {
            cur = cur->_right;
        }
        else if (key < cur->_kv.first) {
            cur = cur->_left;
        }
        else {
            return cur;
        }
    }
    return nullptr;
}
```

补充：递归求出左右子树的大小：

```
  size_t _Size(Node* root) {
      return root == nullptr ? 0 : _Size(root->_left) + _Size(root->_right)
+ 1;
  }
```

6. 最后测试一下

1. 创建一棵包含特殊情况的二叉树进行测试
2. 一堆随机值进行测试

```
void TestAVLTree1()
{
    AVLTree<int, int> t;
    // 常规的测试用例
    //int a[] = { 16, 3, 7, 11, 9, 26, 18, 14, 15 };
    // 特殊的带有双旋场景的测试用例
    int a[] = { 4, 2, 6, 1, 3, 5, 15, 7, 16, 14 };
    for (auto e : a)
    {
        //if (e == 14)
        //{
        //  int x = 0;
        //}手写条件断点用于调试

        t.Insert({ e, e });
        //t.InOrder();
        cout << "Insert:" << e << "->" << t.IsBalanceTree() << endl;
    }
    t.InOrder();
    cout << t.IsBalanceTree() << endl;
}

// 插入一堆随机值，测试平衡，顺便测试一下高度和性能等
void TestAVLTree2()
{
    const int N = 1000000;
    vector<int> v;
    v.reserve(N);
    srand(time(0));
    for (size_t i = 0; i < N; i++)
    {
        v.push_back(rand() + i);
    }

    size_t begin2 = clock();
    AVLTree<int, int> t;
    for (auto e : v)
    {
        t.Insert(make_pair(e, e));
    }
    size_t end2 = clock();
    cout << "Insert:" << end2 - begin2 << endl;
    cout << t.IsBalanceTree() << endl;
    cout << "Height:" << t.Height() << endl;
    cout << "Size:" << t.Size() << endl;
```

```
    size_t begin1 = clock();
    // 确定在的值
    /*for (auto e : v)
    {
    t.Find(e);
    }*/
    // 随机值
    for (size_t i = 0; i < N; i++)
    {
        t.Find((rand() + i));
    }
    size_t end1 = clock();

    cout << "Find:" << end1 - begin1 << endl;
}
```